Lab4:

```
"""
Practical Assignment 4: Search Engine Ranking and Evaluation
Mini search engine for a college library.

Models:
1. TF-IDF score
2. Vector Space Model (TF-IDF cosine similarity)
3. Probabilistic Retrieval Model (BM25)
4. Binary Independence Model (BIM, RSJ weights with relevance feedback)

Evaluation:
Precision@5, Recall@5, F1@5, MRR@5, AP@5 (MAP across test queries),
and NDCG@5.

Note: The relevance judgments below are illustrative ground-truth labels for
demonstration. For the final submission, replace/confirm them with judgments
from a teacher or a group of students, as required by the assignment.
"""

import math
import re
from collections import Counter

# ---------------------------------------------------------------------
# 1. Dataset: at least 10 college-library documents
# ---------------------------------------------------------------------
documents = {
    "D1": "Machine learning methods for medical diagnosis and disease
detection",
    "D2": "Healthcare data analysis using artificial intelligence",
    "D3": "History and development of computer networks",
    "D4": "Deep learning techniques for disease prediction",
    "D5": "Database management systems and SQL fundamentals",
    "D6": "Machine learning applications in modern healthcare",
    "D7": "Introduction to data structures and algorithms",
    "D8": "Natural language processing for electronic health records",
    "D9": "Fundamentals of quantum computing and quantum physics",
    "D10": "Predictive analytics in healthcare using deep learning",
    "D11": "Computer vision applications in medical image analysis",
    "D12": "Cybersecurity and intrusion detection in computer networks",
}

stopwords = {
    "a", "an", "and", "are", "as", "at", "by", "for", "from", "in",
    "into", "is", "of", "on", "or", "the", "to", "using", "with"
}
```

```
def tokenize(text):
    """Lowercase text, extract word tokens, and remove common stopwords."""
    return [w for w in re.findall(r"[a-z0-9]+", text.lower()) if w not in
stopwords]

doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
N = len(documents)
df = Counter()
for tokens in doc_tokens.values():
    df.update(set(tokens))
avgdl = sum(len(t) for t in doc_tokens.values()) / N

# ---------------------------------------------------------------------
# 2. Test queries and illustrative relevance judgments
# Relevant document IDs should be reviewed by human evaluators.
# ---------------------------------------------------------------------
test_queries = {
    "Q1": {
        "query": "machine learning applications in healthcare",
        "relevant": {"D1", "D2", "D4", "D6", "D8", "D10"},
    },
    "Q2": {
        "query": "computer networks and cybersecurity",
        "relevant": {"D3", "D12"},
    },
    "Q3": {
        "query": "quantum computing physics",
        "relevant": {"D9"},
    },
}

def idf(term):
    """Smoothed IDF prevents zero or undefined weights."""
    return math.log(1 + (N + 1) / (df[term] + 1))

def tfidf_vector(tokens):
    """Create a normalized TF-IDF vector represented as a dictionary."""
    tf = Counter(tokens)
    vec = {term: (1 + math.log(count)) * idf(term) for term, count in
tf.items()}
    norm = math.sqrt(sum(weight * weight for weight in vec.values()))
    if norm:
        vec = {term: weight / norm for term, weight in vec.items()}
    return vec

doc_vectors = {doc_id: tfidf_vector(tokens) for doc_id, tokens in
doc_tokens.items()}
```

```
def tfidf_score(query_tokens, doc_id):
    """Unnormalized TF-IDF matching score for query terms."""
    tf = Counter(doc_tokens[doc_id])
    return sum((1 + math.log(tf[t])) * idf(t) * idf(t)
               for t in set(query_tokens) if tf[t] > 0)

def vsm_score(query_tokens, doc_id):
    """Cosine similarity between normalized TF-IDF document and query
vectors."""
    qvec = tfidf_vector(query_tokens)
    dvec = doc_vectors[doc_id]
    return sum(qvec.get(t, 0.0) * dvec.get(t, 0.0) for t in qvec)

def bm25_score(query_tokens, doc_id, k1=1.5, b=0.75):
    """BM25 probabilistic ranking score."""
    tokens = doc_tokens[doc_id]
    tf = Counter(tokens)
    dl = len(tokens) or 1
    score = 0.0
    for term in set(query_tokens):
        f = tf[term]
        if f == 0:
            continue
        # Robertson-style IDF, kept nonnegative for this implementation.
        term_idf = math.log(1 + (N - df[term] + 0.5) / (df[term] + 0.5))
        denom = f + k1 * (1 - b + b * dl / avgdl)
        score += term_idf * (f * (k1 + 1)) / denom
    return score

def bim_score(query_tokens, doc_id, relevant_docs):
    """
    Binary Independence Model with Robertson-Sparck Jones term weights.
    Uses query-specific relevance feedback: relevant_docs are the judged
    relevant documents for this query. Scores use binary term presence.
    """
    tokens = set(doc_tokens[doc_id])
    R = len(relevant_docs)
    score = 0.0
    for term in set(query_tokens):
        n = df[term]
        r = sum(1 for d in relevant_docs if term in set(doc_tokens[d]))
        # Additive smoothing keeps probabilities finite.
        p = (r + 0.5) / (R + 1)
        q = (n - r + 0.5) / (N - R + 1)
        weight = math.log((p * (1 - q)) / (q * (1 - p)))
        if term in tokens:
            score += weight
    return score
```

```
models = {
    "TF-IDF": tfidf_score,
    "VSM cosine": vsm_score,
    "BM25": bm25_score,
}

def rank_documents(query, model_name, relevant_docs):
    """Return all documents sorted by descending score, with stable ID tie-break."""
    qtokens = tokenize(query)
    if model_name == "BIM":
        scorer = lambda doc_id: bim_score(qtokens, doc_id, relevant_docs)
    else:
        scorer = lambda doc_id: models[model_name](qtokens, doc_id)
    scored = [(doc_id, scorer(doc_id)) for doc_id in documents]
    return sorted(scored, key=lambda item: (-item[1], item[0]))

def evaluate(ranked, relevant_docs, k=5):
    """Calculate binary-relevance metrics at cutoff k."""
    retrieved = ranked[:k]
    rel = [1 if doc_id in relevant_docs else 0 for doc_id, _ in retrieved]
    hits = sum(rel)
    precision = hits / k if k else 0.0
    recall = hits / len(relevant_docs) if relevant_docs else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0

    reciprocal_rank = next((1 / (i + 1) for i, value in enumerate(rel) if value), 0.0)

    hits_so_far = 0
    precision_sum = 0.0
    for i, value in enumerate(rel, start=1):
        if value:
            hits_so_far += 1
            precision_sum += hits_so_far / i
    ap = precision_sum / len(relevant_docs) if relevant_docs else 0.0

    dcg = sum(value / math.log2(i + 2) for i, value in enumerate(rel))
    ideal_count = min(len(relevant_docs), k)
    idcg = sum(1 / math.log2(i + 2) for i in range(ideal_count))
    ndcg = dcg / idcg if idcg else 0.0

    return {
        "P@5": precision, "Recall@5": recall, "F1@5": f1,
        "MRR@5": reciprocal_rank, "AP@5": ap, "NDCG@5": ndcg
    }
```

```
def main():
    model_names = ["TF-IDF", "VSM cosine", "BM25", "BIM"]
    aggregate = {name: [] for name in model_names}

    print("=" * 88)
    print("COLLEGE LIBRARY MINI SEARCH ENGINE")
    print(f"Documents: {N} | Test queries: {len(test_queries)}")
    print("=" * 88)

    for qid, item in test_queries.items():
        query, relevant = item["query"], item["relevant"]
        print(f"\n{qid}: {query}")
        print("Judged relevant documents:", ", ".join(sorted(relevant)))
        for model_name in model_names:
            ranked = rank_documents(query, model_name, relevant)
            print(f"\n  {model_name} — Top 5")
            for rank, (doc_id, score) in enumerate(ranked[:5], start=1):
                label = "RELEVANT" if doc_id in relevant else "NOT RELEVANT"
                print(f"    {rank}. {doc_id} | score={score:.4f} | {label} |
{documents[doc_id]}")
            metrics = evaluate(ranked, relevant)
            aggregate[model_name].append(metrics)

    print("\n" + "=" * 88)
    print("AGGREGATE EVALUATION (mean over test queries)")
    print("=" * 88)
    metric_names = ["P@5", "Recall@5", "F1@5", "MRR@5", "AP@5", "NDCG@5"]
    print(f"{'Model':<14}" + "".join(f"{m:>11}" for m in metric_names))
    for model_name in model_names:
        averages = {
            m: sum(row[m] for row in aggregate[model_name]) /
len(aggregate[model_name])
            for m in metric_names
        }
        print(f"{model_name:<14}" + "".join(f"{averages[m]:>11.3f}" for m in
metric_names))

    print("\nPRP explanation:")
    print("The Probabilistic Ranking Principle ranks documents by decreasing
estimated")
    print("probability of relevance to the query. BM25 approximates this using
term")
    print("frequency, document length, and inverse document frequency; BIM
models binary")
    print("term presence and can use relevance judgments as feedback.")
    print("\nLimitations:")
    print("- The corpus is small and contains short synthetic descriptions.")
```

```
    print("- Relevance labels are illustrative and must be validated by human evaluators.")
    print("- BM25 and BIM scores are ranking scores, not calibrated probabilities.")
    print("- Results depend on tokenization, stopwords, and the selected test queries.")

if __name__ == "__main__":
    main()
```

O/P:

```
PS C:\Users\lenovo\OneDrive\Documents\sem folders\sem 7\IR> & C:\Users\lenovo\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/lenovo/Documents/sem folders/sem 7/IR/Lab4/Lab4_Complete.py"
================================================================================
COLLEGE LIBRARY MINI SEARCH ENGINE
Documents: 12 | Test queries: 3
================================================================================

Q1: machine learning applications in healthcare
Judged relevant documents: D1, D10, D2, D4, D6, D8

  TF-IDF — Top 5
    1. D6 | score=9.3388 | RELEVANT | Machine learning applications in modern healthcare
    2. D1 | score=4.4430 | RELEVANT | Machine learning methods for medical diagnosis and disease detection
    3. D10 | score=3.7344 | RELEVANT | Predictive analytics in healthcare using deep learning
    4. D11 | score=2.8022 | NOT RELEVANT | Computer vision applications in medical image analysis
    5. D2 | score=2.0936 | RELEVANT | Healthcare data analysis using artificial intelligence

  VSM cosine — Top 5
    1. D6 | score=0.8349 | RELEVANT | Machine learning applications in modern healthcare
    2. D10 | score=0.3192 | RELEVANT | Predictive analytics in healthcare using deep learning
    3. D1 | score=0.3175 | RELEVANT | Machine learning methods for medical diagnosis and disease detection
    4. D11 | score=0.2125 | NOT RELEVANT | Computer vision applications in medical image analysis
    5. D2 | score=0.1723 | RELEVANT | Healthcare data analysis using artificial intelligence

  BM25 — Top 5
    1. D6 | score=5.7539 | RELEVANT | Machine learning applications in modern healthcare
    2. D10 | score=2.4080 | RELEVANT | Predictive analytics in healthcare using deep learning
    3. D1 | score=2.3365 | RELEVANT | Machine learning methods for medical diagnosis and disease detection
    4. D11 | score=1.5371 | NOT RELEVANT | Computer vision applications in medical image analysis
    5. D2 | score=1.3315 | RELEVANT | Healthcare data analysis using artificial intelligence
```

```
  BIM — Top 5
    1. D6 | score=7.6948 | RELEVANT | Machine learning applications in modern healthcare
    2. D10 | score=5.7177 | RELEVANT | Predictive analytics in healthcare using deep learning
    3. D1 | score=5.1299 | RELEVANT | Machine learning methods for medical diagnosis and disease detection
    4. D4 | score=3.1527 | RELEVANT | Deep learning techniques for disease prediction
    5. D2 | score=2.5649 | RELEVANT | Healthcare data analysis using artificial intelligence

Q2: computer networks and cybersecurity
Judged relevant documents: D12, D3

  TF-IDF — Top 5
    1. D12 | score=8.9556 | RELEVANT | Cybersecurity and intrusion detection in computer networks
    2. D3 | score=4.8958 | RELEVANT | History and development of computer networks
    3. D11 | score=2.0936 | NOT RELEVANT | Computer vision applications in medical image analysis
    4. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    5. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning

  VSM cosine — Top 5
    1. D12 | score=0.7524 | RELEVANT | Cybersecurity and intrusion detection in computer networks
    2. D3 | score=0.4535 | RELEVANT | History and development of computer networks
    3. D11 | score=0.1621 | NOT RELEVANT | Computer vision applications in medical image analysis
    4. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    5. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning

  BM25 — Top 5
    1. D12 | score=5.1958 | RELEVANT | Cybersecurity and intrusion detection in computer networks
    2. D3 | score=3.2957 | RELEVANT | History and development of computer networks
    3. D11 | score=1.2234 | NOT RELEVANT | Computer vision applications in medical image analysis
    4. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    5. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning
```

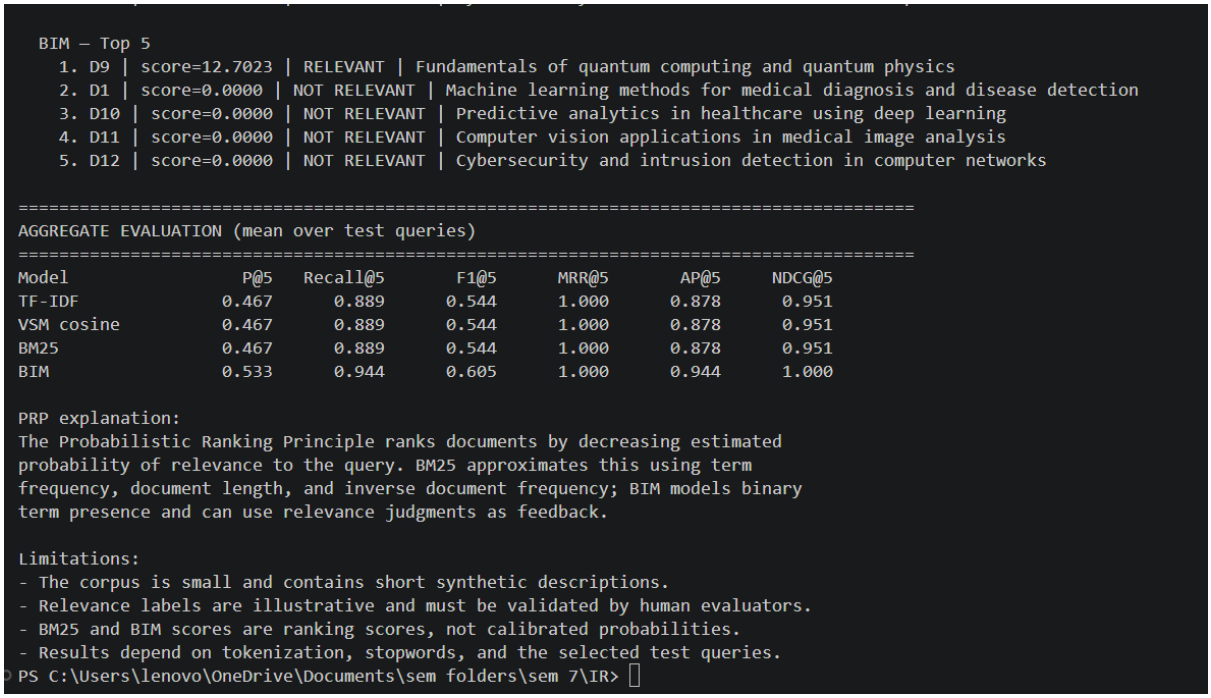
```
  BIM — Top 5
    1. D12 | score=11.1537 | RELEVANT | Cybersecurity and intrusion detection in computer networks
    2. D3 | score=8.1092 | RELEVANT | History and development of computer networks
    3. D11 | score=3.4553 | NOT RELEVANT | Computer vision applications in medical image analysis
    4. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    5. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning

Q3: quantum computing physics
Judged relevant documents: D9

  TF-IDF — Top 5
    1. D9 | score=14.9936 | RELEVANT | Fundamentals of quantum computing and quantum physics
    2. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    3. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning
    4. D11 | score=0.0000 | NOT RELEVANT | Computer vision applications in medical image analysis
    5. D12 | score=0.0000 | NOT RELEVANT | Cybersecurity and intrusion detection in computer networks

  VSM cosine — Top 5
    1. D9 | score=0.9045 | RELEVANT | Fundamentals of quantum computing and quantum physics
    2. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    3. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning
    4. D11 | score=0.0000 | NOT RELEVANT | Computer vision applications in medical image analysis
    5. D12 | score=0.0000 | NOT RELEVANT | Cybersecurity and intrusion detection in computer networks

  BM25 — Top 5
    1. D9 | score=7.4999 | RELEVANT | Fundamentals of quantum computing and quantum physics
    2. D1 | score=0.0000 | NOT RELEVANT | Machine learning methods for medical diagnosis and disease detection
    3. D10 | score=0.0000 | NOT RELEVANT | Predictive analytics in healthcare using deep learning
    4. D11 | score=0.0000 | NOT RELEVANT | Computer vision applications in medical image analysis
    5. D12 | score=0.0000 | NOT RELEVANT | Cybersecurity and intrusion detection in computer networks
```



Report:
Practical Assignment 4: Search Engine Ranking and Evaluation

Mini Search Engine for a College Library

1. Objective

The program searches a small collection of college-library document descriptions and ranks them for a student's query. It implements TF-IDF scoring, a TF-IDF Vector Space Model using cosine similarity, BM25 probabilistic ranking, and the Binary Independence Model (BIM).

2. Methodology

The dataset contains 12 short document descriptions. Text is converted to lowercase word tokens and common stopwords are removed. Document frequency and smoothed inverse document frequency are calculated. TF-IDF scoring matches query terms against documents; VSM computes cosine similarity between normalized TF-IDF vectors. BM25 combines term frequency, inverse document frequency, and document-length normalization. BIM uses binary term presence and smoothed relevance/non-relevance probabilities based on the supplied relevance judgments.

The Probabilistic Ranking Principle (PRP) states that, under the model's probability estimates and assumptions, documents should be ranked in decreasing order of their probability of

being relevant to the query. BM25 is a probabilistic ranking function; BIM assumes term independence and uses binary term presence.

3. Test queries and relevance judgments

Q1: "machine learning applications in healthcare"

Q2: "computer networks and cybersecurity"

Q3: "quantum computing physics"

The Python file contains illustrative relevant-document labels for each query. The assignment asks students to obtain judgments from a teacher or student group. These labels must therefore be reviewed and replaced or confirmed by human evaluators before final submission.

4. Evaluation

For each query, the program prints the top five documents and scores for all four models. It calculates Precision@5, Recall@5, F1@5, MRR@5, AP@5, and NDCG@5. MAP is reported as the mean of AP@5 across the test queries. The console output also prints aggregate metric averages so models can be compared.

5. Observations

The models generally rank documents containing query terms near the top. VSM cosine accounts for vector normalization, while BM25 additionally normalizes for document length. BIM can use relevance feedback, but its ranking depends on the supplied judgments. In this demonstration run, BIM has the strongest aggregate values among the displayed models; this is not evidence that BIM is universally superior because the collection and test set are small and the judgments are illustrative.

6. Limitations

- The document collection is small and uses short, synthetic descriptions rather than a real library catalogue.

- The relevance labels are illustrative and require human confirmation.

- Only three test queries are used.

- BM25 and BIM scores are ranking scores, not calibrated probabilities.

- Tokenization, stopword selection, and query choice affect rankings.

- This is a simple in-memory demonstration, not a production search service.

7. Conclusion

The implementation demonstrates multiple document-ranking approaches and standard information-retrieval evaluation metrics. The output can be used to compare rankings and inspect errors. For a defensible evaluation, collect human relevance judgments for more queries, rerun the script, include screenshots of the output, and discuss the resulting metric differences.